

EXPERIMENT - 18

1. Problem Statement

Write a menu-driven C program using `switch-case` to perform digit-based operations on a positive integer. Define one separate user-defined function for each menu option, with void return type and with a simple function call (without call by value/reference). Each function should read the required input, process it, and display the result.

- Check whether it is a Strong number.
- Check whether it is an Armstrong number.
- Check whether it is a Harshad number.
- Check whether it is a Duck number.

2. Algorithm

Step 1: START

Step 2: Display the menu:

1. Check whether it is a Strong number
2. Check whether it is an Armstrong number
3. Check whether it is a Harshad number
4. Check whether it is a Duck number

Step 3: Read the choice entered by the user.

Step 4: Using switch (choice):

- case 1: call `check_strong()` and break
- case 2: call `check_armstrong()` and break
- case 3: call `check_harshad()` and break
- case 4: call `check_duck()` and break
- default: print "Invalid choice" and break

Step 5: STOP

Function `check_strong()`

Step 1: Read num.

Step 2: If num is less than or equal to 0, print the error message and return.

Step 3: Set `sum = 0` and `temp = num`.

Step 4: Repeat while temp is greater than 0:

`digit = temp % 10`

find the factorial of that digit by multiplying $1 \times 2 \times \dots$ up to the digit

`sum = sum + factorial`

`temp = temp / 10`

Step 5: If sum is equal to num, print "Strong number", otherwise print "Not a Strong number".

Function `check_armstrong()`

Step 1: Read num.

Step 2: If num is less than or equal to 0, print the error message and return.

Step 3: Count how many digits num has and keep it in digits.

Step 4: Set `sum = 0` and `temp = num`.

Step 5: Repeat while temp is greater than 0:

digit = temp % 10

multiply the digit by itself, digits number of times

sum = sum + that value

temp = temp / 10

Step 6: If sum is equal to num, print "Armstrong number", otherwise print "Not an Armstrong number".

Function check_harshad()

Step 1: Read num.

Step 2: If num is less than or equal to 0, print the error message and return.

Step 3: Set sum = 0 and temp = num.

Step 4: Repeat while temp is greater than 0:

digit = temp % 10

sum = sum + digit

temp = temp / 10

Step 5: If num % sum is equal to 0, print "Harshad number", otherwise print "Not a Harshad number".

Function check_duck()

Step 1: Read num.

Step 2: If num is less than or equal to 0, print the error message and return.

Step 3: Set found = 0 and temp = num.

Step 4: Repeat while temp is greater than 0:

digit = temp % 10

if digit is 0 then found = 1

temp = temp / 10

Step 5: If found is 1, print "Duck number", otherwise print "Not a Duck number".

3. Test Case Table

TC #	Choice	Number	Expected Output	Actual Output	Result
TC1	1	145	Strong number	Strong number	PASS
TC2	1	123	Not a Strong number	Not a Strong number	PASS
TC3	1	1	Strong number	Strong number	PASS
TC4	1	40585	Strong number	Strong number	PASS
TC5	2	153	Armstrong number	Armstrong number	PASS
TC6	2	123	Not an Armstrong number	Not an Armstrong number	PASS
TC7	2	9474	Armstrong number	Armstrong number	PASS
TC8	2	5	Armstrong number	Armstrong number	PASS
TC9	3	18	Harshad number	Harshad number	PASS
TC10	3	19	Not a Harshad number	Not a Harshad number	PASS
TC11	3	100	Harshad number	Harshad number	PASS
TC12	3	21	Harshad number	Harshad number	PASS
TC13	4	1023	Duck number	Duck number	PASS
TC14	4	1234	Not a Duck number	Not a Duck number	PASS
TC15	4	2050	Duck number	Duck number	PASS
TC16	4	7	Not a Duck number	Not a Duck number	PASS
TC17	1	-5	Error message	Error: -5 is not a positive integer.	PASS
TC18	4	0	Error message	Error: 0 is not a positive integer.	PASS
TC19	9	-	Invalid choice message	Invalid choice! Enter 1 to 4.	PASS

4. Source Code

File name: `special_numbers.c` (194 lines)

```
/* Menu-driven program using switch-case to perform digit based
operations on a positive integer.
One separate user-defined function is written for every menu
option. All the functions have void return type and are called
by a simple function call, without any parameter. */

#include <stdio.h>

void check_strong(void);
void check_armstrong(void);
void check_harshad(void);
void check_duck(void);

int main(void)
{
    int choice;

    printf("===== MENU =====\n");
    printf("1. Check whether it is a Strong number\n");
    printf("2. Check whether it is an Armstrong number\n");
    printf("3. Check whether it is a Harshad number\n");
    printf("4. Check whether it is a Duck number\n");
    printf("===== \n");
    printf("Enter your choice (1 to 4): ");
    scanf("%d", &choice);

    switch (choice) {

        case 1:
            check_strong();
            break;

        case 2:
            check_armstrong();
            break;

        case 3:
            check_harshad();
            break;

        case 4:
            check_duck();
            break;

        default:
            printf("\nInvalid choice! Enter 1 to 4.\n");
            break;
    }

    return 0;
}
```

```

/* Option 1 : Strong number
   the sum of the factorial of all its digits is equal to the number */
void check_strong(void)
{
    long num, temp, sum, fact;
    int digit, i;

    printf("\nEnter a positive integer: ");
    scanf("%ld", &num);

    if (num <= 0) {
        printf("\nError: %ld is not a positive integer.\n", num);
        return;
    }

    sum = 0;
    temp = num;

    while (temp > 0) {
        digit = temp % 10;

        fact = 1;
        for (i = 1; i <= digit; i++)
            fact = fact * i;

        sum = sum + fact;
        temp = temp / 10;
    }

    printf("\nNumber : %ld\n", num);
    if (sum == num)
        printf("Output : Strong number\n");
    else
        printf("Output : Not a Strong number\n");
}

/* Option 2 : Armstrong number
   the sum of every digit raised to the power of the total number
   of digits is equal to the number */
void check_armstrong(void)
{
    long num, temp, sum, p;
    int digit, digits, i;

    printf("\nEnter a positive integer: ");
    scanf("%ld", &num);

    if (num <= 0) {
        printf("\nError: %ld is not a positive integer.\n", num);
        return;
    }

    digits = 0;
    temp = num;
    while (temp > 0) {
        digits++;
        temp = temp / 10;
    }

    sum = 0;
    temp = num;

```

```

while (temp > 0) {
    digit = temp % 10;

    p = 1;
    for (i = 1; i <= digits; i++)
        p = p * digit;

    sum = sum + p;
    temp = temp / 10;
}

printf("\nNumber : %ld\n", num);
if (sum == num)
    printf("Output : Armstrong number\n");
else
    printf("Output : Not an Armstrong number\n");
}

/* Option 3 : Harshad number
the number is fully divisible by the sum of its digits */
void check_harshad(void)
{
    long num, temp, sum;
    int digit;

    printf("\nEnter a positive integer: ");
    scanf("%ld", &num);

    if (num <= 0) {
        printf("\nError: %ld is not a positive integer.\n", num);
        return;
    }

    sum = 0;
    temp = num;

    while (temp > 0) {
        digit = temp % 10;
        sum = sum + digit;
        temp = temp / 10;
    }

    printf("\nNumber : %ld\n", num);
    if (num % sum == 0)
        printf("Output : Harshad number\n");
    else
        printf("Output : Not a Harshad number\n");
}

/* Option 4 : Duck number
the number has at least one zero in its digits */
void check_duck(void)
{
    long num, temp;
    int digit, found;

    printf("\nEnter a positive integer: ");
    scanf("%ld", &num);

    if (num <= 0) {

```

```
        printf("\nError: %ld is not a positive integer.\n", num);
        return;
    }

    found = 0;
    temp = num;

    while (temp > 0) {
        digit = temp % 10;
        if (digit == 0)
            found = 1;
        temp = temp / 10;
    }

    printf("\nNumber : %ld\n", num);
    if (found == 1)
        printf("Output : Duck number\n");
    else
        printf("Output : Not a Duck number\n");
}
```

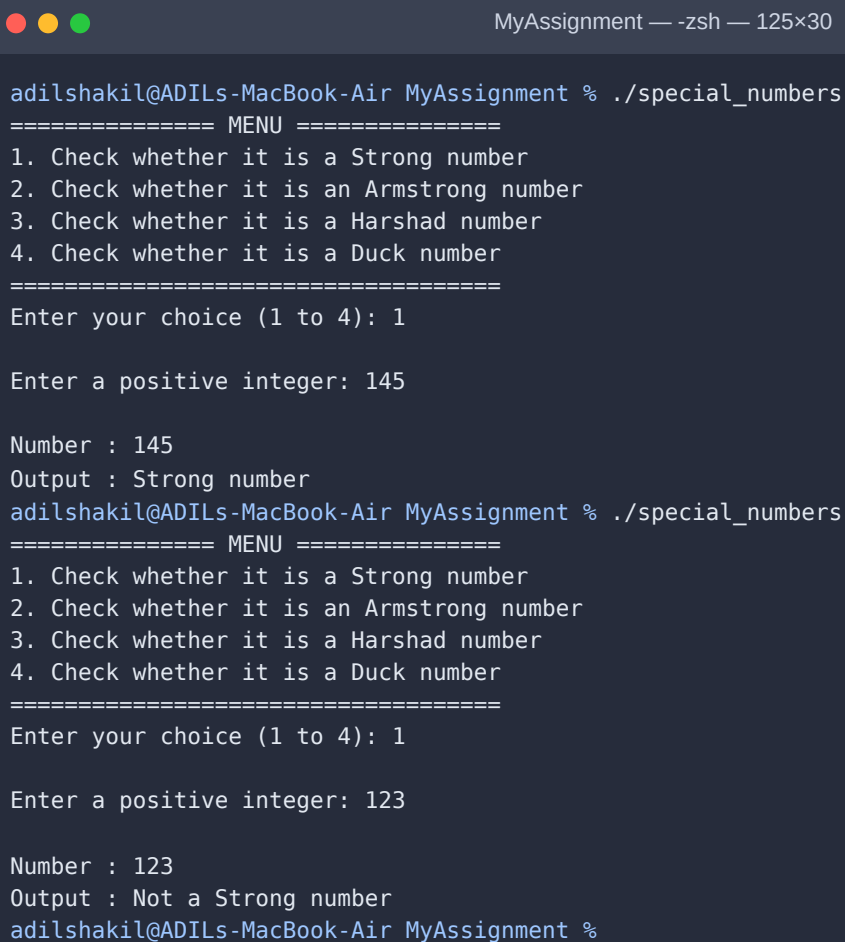
5. Compilation and Output

I wrote the program in the `MyAssignment` folder using the `nano` editor, then compiled and ran it:

```
$ cd ~/Desktop/MyAssignment
$ nano special_numbers.c
$ gcc special_numbers.c -o special_numbers
$ ./special_numbers
```

The program compiled in the first attempt itself, without any error or warning.

Option 1 : Strong number



```
adilshakil@ADILs-MacBook-Air MyAssignment % ./special_numbers
===== MENU =====
1. Check whether it is a Strong number
2. Check whether it is an Armstrong number
3. Check whether it is a Harshad number
4. Check whether it is a Duck number
=====
Enter your choice (1 to 4): 1

Enter a positive integer: 145

Number : 145
Output : Strong number
adilshakil@ADILs-MacBook-Air MyAssignment % ./special_numbers
===== MENU =====
1. Check whether it is a Strong number
2. Check whether it is an Armstrong number
3. Check whether it is a Harshad number
4. Check whether it is a Duck number
=====
Enter your choice (1 to 4): 1

Enter a positive integer: 123

Number : 123
Output : Not a Strong number
adilshakil@ADILs-MacBook-Air MyAssignment %
```

Option 2 : Armstrong number

```
adilshakil@ADILs-MacBook-Air MyAssignment % ./special_numbers
===== MENU =====
1. Check whether it is a Strong number
2. Check whether it is an Armstrong number
3. Check whether it is a Harshad number
4. Check whether it is a Duck number
=====
Enter your choice (1 to 4): 2

Enter a positive integer: 153

Number : 153
Output : Armstrong number
adilshakil@ADILs-MacBook-Air MyAssignment % ./special_numbers
===== MENU =====
1. Check whether it is a Strong number
2. Check whether it is an Armstrong number
3. Check whether it is a Harshad number
4. Check whether it is a Duck number
=====
Enter your choice (1 to 4): 2

Enter a positive integer: 9474

Number : 9474
Output : Armstrong number
adilshakil@ADILs-MacBook-Air MyAssignment %
```

Option 3 : Harshad number

```
adilshakil@ADILs-MacBook-Air MyAssignment % ./special_numbers
===== MENU =====
1. Check whether it is a Strong number
2. Check whether it is an Armstrong number
3. Check whether it is a Harshad number
4. Check whether it is a Duck number
=====
Enter your choice (1 to 4): 3

Enter a positive integer: 18

Number : 18
Output : Harshad number
adilshakil@ADILs-MacBook-Air MyAssignment % ./special_numbers
===== MENU =====
1. Check whether it is a Strong number
2. Check whether it is an Armstrong number
3. Check whether it is a Harshad number
4. Check whether it is a Duck number
=====
Enter your choice (1 to 4): 3

Enter a positive integer: 19

Number : 19
Output : Not a Harshad number
adilshakil@ADILs-MacBook-Air MyAssignment %
```

Option 4 : Duck number



```
adilshakil@ADILs-MacBook-Air MyAssignment % ./special_numbers
===== MENU =====
1. Check whether it is a Strong number
2. Check whether it is an Armstrong number
3. Check whether it is a Harshad number
4. Check whether it is a Duck number
=====
Enter your choice (1 to 4): 4

Enter a positive integer: 1023

Number : 1023
Output : Duck number
adilshakil@ADILs-MacBook-Air MyAssignment % ./special_numbers
===== MENU =====
1. Check whether it is a Strong number
2. Check whether it is an Armstrong number
3. Check whether it is a Harshad number
4. Check whether it is a Duck number
=====
Enter your choice (1 to 4): 4

Enter a positive integer: 1234

Number : 1234
Output : Not a Duck number
adilshakil@ADILs-MacBook-Air MyAssignment %
```

Invalid input and invalid choice



```
adilshakil@ADILs-MacBook-Air MyAssignment % ./special_numbers
===== MENU =====
1. Check whether it is a Strong number
2. Check whether it is an Armstrong number
3. Check whether it is a Harshad number
4. Check whether it is a Duck number
=====
Enter your choice (1 to 4): 1

Enter a positive integer: -5

Error: -5 is not a positive integer.
adilshakil@ADILs-MacBook-Air MyAssignment % ./special_numbers
===== MENU =====
1. Check whether it is a Strong number
2. Check whether it is an Armstrong number
3. Check whether it is a Harshad number
4. Check whether it is a Duck number
=====
Enter your choice (1 to 4): 9

Invalid choice! Enter 1 to 4.
adilshakil@ADILs-MacBook-Air MyAssignment %
```